

Conceitos de Sistemas Operacionais

- Examinaremos alguns conceitos básicos de SO, usando exemplos de UNIX/Linux de forma resumida, para uma introdução:
 - Processos
 - Memória
 - Arquivos
 - Entrada e Saída
 - Proteção

Processos

- Definição: “Programa em execução + recursos alocados”
- Associado aos processos temos:
 - Espaço de endereçamento
 - Registradores, arquivos abertos, estados
- Uma visão intuitiva de processo é pensar numa sistema de Multiprogramação (editor de texto + navegador + leitor de emails)

Estados de um Processo

- Execução → Pronto → Bloqueado
- Tabela de processos: armazena as informações dos processos existentes no momento.
- Alguns conceitos relacionados aos processos:
 - Processos filhos (hierarquia em árvore)
 - Comunicação entre processos
 - Sinal de alarme – mensagem enviada pelo SO para executar alguma tarefa
 - PID, UID, GID.

Espaço de endereçamento

- Memória física vs Virtual:
 - 32/64 bits (ex: 2^{32} endereços)
 - Técnica de paginação (disco + RAM)

Arquivos e diretórios

- Sistema de Arquivos:
 - Hierarquia (ex: /home/usuário/docs)
 - Operações: criar, ler, escrever (através de chamadas de sistemas)
 - Hierarquia de diretórios em árvore
 - Nome de caminho: a partir do diretório raiz (/ ou \).
 - Diretório de trabalho
 - Descritor de arquivo
- Montagem: integração de dispositivos (ex: CD-ROM em /mnt)

Arquivos especiais e pipe

- Arquivos Especiais: `/dev/lp` (impressora se parece com um arquivo).
- Pipes: Comunicação entre processos (ex: `cat file | sort`).

Entrada/Saída

- Todos os computadores tem dispositivos de E/S que cabem ao S.O. gerenciar.
 - Teclados, monitores, impressores, mouse
- Todo SO tem um subsistema de E/S para gerenciar os dispositivos de E/S:
 - Alguns são independentes
 - Outros, como drivers de dispositivos, são específicos

Proteção

- Bits UNIX (rwx):
 - Ex: rwxr-x--x → Dono (leitura/escrita/execução), grupo (leitura e execução) e outros(só execução)
- Superusuário: Permissões administrativas.

Shell (Interpretador de Comandos)

- SO é o código que executa chamadas de sistema. Shell, compilador, editores, programas utilitários não fazem parte do sistema operacional
- Função do Shell: Interface usuário-SO. (algumas sh, csh, ksh, bash)
- Comandos:
 - Redirecionamento (date > file).
 - Pipes (cat file1 | sort).
 - Background (comando &).

Conceitos obsoletos

- Conceitos podem se tornar obsoletos com o desenvolvimento de tecnologia. Por exemplo, cache pode desaparecer com o desenvolvimento de memórias mais rápidas.
- Mudanças de tecnologias podem trazer de volta alguns chamados conceitos obsoletos
- Alguns exemplos:
 - Tamanho da memória (linguagens e linguagens de montagem)
 - Hardware de proteção (monoprogramação/multiprogramação)
 - Discos (um único diretório, vários diretórios)
 - Memória Virtual (capacidade de executar programas maiores que a memória)

Chamadas de Sistema

Gerenciamento de processos

Chamada	Descrição
<code>pid = fork()</code>	Cria um processo filho idêntico ao pai
<code>pid = waitpid(pid, &statloc, options)</code>	Espera que um processo filho seja concluído
<code>s = execve(name, argv, environp)</code>	Substitui a imagem do núcleo de um processo
<code>exit(status)</code>	Conclui a execução do processo e devolve status

Interpretador de Comandos (Simples)

```
#define TRUE 1
```

```
while (TRUE) {  
    type_prompt( );  
    read_command(command, parameters);  
  
    if (fork() != 0) {  
        /*Codigo do processo pai. */  
        waitpid(-1, &status, 0);  
    } else {  
        /*Codigo do processo filho. */  
        execve(command, parameters, 0);  
    }  
}
```

```
/* repita para sempre */
```

```
/* mostra prompt na tela */
```

```
/* le entrada do terminal */
```

```
/* cria processo filho */
```

```
/* aguarda o processo filho acabar */
```

```
/* executa o comando */
```

Chamadas de Sistema

Gerenciamento de arquivos

Chamada	Descrição
<code>fd = open(file, how, ...)</code>	Abre um arquivo para leitura, escrita ou ambos
<code>s = close(fd)</code>	Fecha um arquivo aberto
<code>n = read(fd, buffer, nbytes)</code>	Lê dados a partir de um arquivo em um buffer
<code>n = write(fd, buffer, nbytes)</code>	Escreve dados a partir de um buffer em um arquivo
<code>position = lseek(fd, offset, whence)</code>	Move o ponteiro do arquivo
<code>s = stat(name, &buf)</code>	Obtém informações sobre um arquivo

Chamadas de Sistema

Gerenciamento do sistema de diretório e arquivo

Chamada	Descrição
s = mkdir(name, mode)	Cria um novo diretório
s = rmdir(name)	Remove um diretório vazio
s = link(name1, name2)	Cria uma nova entrada, name2, apontando para name1
s = unlink(name)	Remove uma entrada de diretório
s = mount(special, name, flag)	Monta um sistema de arquivos
s = umount(special)	Desmonta um sistema de arquivos

`link("/usr/jim/memo", "/usr/ast/note");`

/usr/ast		/usr/jim	
16	correio	31	bin
81	jogos	70	memo
40	teste	59	f.c.
		38	prog1

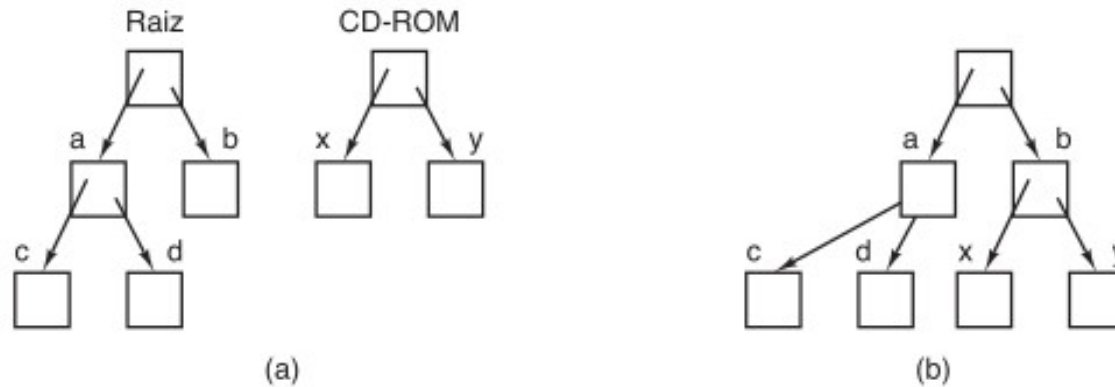
(a)

/usr/ast		/usr/jim	
16	correio	31	bin
81	jogos	70	memo
40	teste	59	f.c.
70	nota	38	prog1

(b)

mount

FIGURA 1.15 (a) Antes da montagem, os arquivos no CD-ROM não estão acessíveis. (b) Depois da montagem, eles fazem parte da hierarquia de arquivos.



Chamadas de Sistema

Diversas

Chamada	Descrição
s = chdir(dirname)	Altera o diretório de trabalho
s = chmod(name, mode)	Altera os bits de proteção de um arquivo
s = kill(pid, signal)	Envia um sinal para um processo
seconds = time(&seconds)	Obtém o tempo decorrido desde 1ª de janeiro de 1970

API Win32

FIGURA 1.23 As chamadas da API Win32 que correspondem aproximadamente às chamadas UNIX da Figura 1.18. Vale a pena enfatizar que o Windows tem um número muito grande de outras chamadas de sistema, a maioria das quais não corresponde a nada no UNIX.

UNIX	Win32	Descrição
fork	CreateProcess	Cria um novo processo
waitpid	WaitForSingleObject	Pode esperar que um processo termine
execve	(nenhuma)	CreateProcess = fork + execve
exit	ExitProcess	Conclui a execução
open	CreateFile	Cria um arquivo ou abre um arquivo existente
close	CloseHandle	Fecha um arquivo
read	ReadFile	Lê dados a partir de um arquivo
write	WriteFile	Escreve dados em um arquivo
lseek	SetFilePointer	Move o ponteiro do arquivo
stat	GetFileAttributesEx	Obtém vários atributos do arquivo
mkdir	CreateDirectory	Cria um novo diretório
rmdir	RemoveDirectory	Remove um diretório vazio
link	(nenhuma)	Win32 não dá suporte a ligações
unlink	DeleteFile	Destrói um arquivo existente
mount	(nenhuma)	Win32 não dá suporte a mount
umount	(nenhuma)	Win32 não dá suporte a mount
chdir	SetCurrentDirectory	Altera o diretório de trabalho atual
chmod	(nenhuma)	Win32 não dá suporte a segurança (embora o NT suporte)
kill	(nenhuma)	Win32 não dá suporte a sinais
time	GetLocalTime	Obtém o tempo atual